P or that appear in one or more relations whose names appear in P. For example, $dom(t \in loan \wedge t[amount] > 1200)$ is the set containing 1200 as well as the set of all values appearing in *loan*. Also, $dom(\neg (t \in loan))$ is the set of all values appearing in *loan*, since the relation $loan$ is mentioned in the expression.

We say that an expression $\{t \mid P(t)\}$ is *safe* if all values that appear in the result are values from $dom(P)$. The expression $\{t \mid \neg (t \in loan)\}$ is not safe. Note that $dom(\neg (t \in loan))$ is the set of all values appearing in *loan*. However, it is possible to have a tuple *t* not in *loan* that contains values that do not appear in *loan*. The other examples of tuple-relational-calculus expressions that we have written in this section are safe.

3.6.4 Expressive Power of Languages

The tuple relational calculus restricted to safe expressions is equivalent in expressive power to the basic relational algebra (with the operators $\cup, -, \times, \sigma$, and ρ, but without the extended relational operators such as generalized projection $\mathcal{G}$ and the outer-join operations) Thus, for every relational-algebra expression using only the basic operations, there is an equivalent expression in the tuple relational calculus, and for every tuple-relational-calculus expression, there is an equivalent relational-algebra expression. We will not prove this assertion here; the bibliographic notes contain references to the proof. Some parts of the proof are included in the exercises. We note that the tuple relational calculus does not have any equivalent of the aggregate operation, but it can be extended to support aggregation. Extending the tuple relational calculus to handle arithmetic expressions is straightforward.

3.7 The Domain Relational Calculus**

A second form of relational calculus, called **domain relational calculus**, uses *domain* variables that take on values from an attributes domain, rather than values for an entire tuple. The domain relational calculus, however, is closely related to the tuple relational calculus.

Domain relational calculus serves as the theoretical basis of the widely used QBE language, just as relational algebra serves as the basis for the SQL language.

3.7.1 Formal Definition

An expression in the domain relational calculus is of the form

$$\{< x_1, x_2, \ldots, x_n > \mid P(x_1, x_2, \ldots, x_n)\}$$

where $x_1, x_2, \ldots, x_n$ represent domain variables. *P* represents a formula composed of atoms, as was the case in the tuple relational calculus. An atom in the domain relational calculus has one of the following forms:

- $< x_1, x_2, \ldots, x_n > \in r$, where *r* is a relation on *n* attributes and $x_1, x_2, \ldots, x_n$ are domain variables or domain constants.

- $x\ \Theta\ y$, where x and y are domain variables and Θ is a comparison operator ($<$, $\leq$, $=$, $\neq$, $>$, $\geq$). We require that attributes x and y have domains that can be compared by Θ.
- $x\ \Theta\ c$, where x is a domain variable, Θ is a comparison operator, and c is a constant in the domain of the attribute for which x is a domain variable.

We build up formulae from atoms by using the following rules:

- An atom is a formula.
- If P_1 is a formula, then so are $\neg P_1$ and (P_1).
- If P_1 and P_2 are formulae, then so are $P_1 \vee P_2$, $P_1 \wedge P_2$, and $P_1 \Rightarrow P_2$.
- If $P_1(x)$ is a formula in x, where x is a domain variable, then

  $$\exists\, x\ (P_1(x)) \text{ and } \forall\, x\ (P_1(x))$$

  are also formulae.

As a notational shorthand, we write

$$\exists\, a, b, c\ (P(a, b, c))$$

for

$$\exists\, a\ (\exists\, b\ (\exists\, c\ (P(a, b, c))))$$

3.7.2 Example Queries

We now give domain-relational-calculus queries for the examples that we considered earlier. Note the similarity of these expressions and the corresponding tuple-relational-calculus expressions.

- Find the loan number, branch name, and amount for loans of over $1200:

  $$\{< l, b, a > \mid < l, b, a > \in \mathit{loan} \wedge a > 1200\}$$

- Find all loan numbers for loans with an amount greater than $1200:

  $$\{< l > \mid \exists\, b, a\ (< l, b, a > \in \mathit{loan} \wedge a > 1200)\}$$

Although the second query appears similar to the one that we wrote for the tuple relational calculus, there is an important difference. In the tuple calculus, when we write $\exists\, s$ for some tuple variable s, we bind it immediately to a relation by writing $\exists\, s \in r$. However, when we write $\exists\, b$ in the domain calculus, b refers not to a tuple, but rather to a domain value. Thus, the domain of variable b is unconstrained until the subformula $< l, b, a > \in \mathit{loan}$ constrains b to branch names that appear in the *loan* relation. For example,

- Find the names of all customers who have a loan from the Perryridge branch and find the loan amount:

$$\{< c, a > \mid \exists\ l\ (< c, l > \in\ borrower \wedge \exists\ b\ (< l, b, a > \in\ loan\ \wedge\ b\ =\ \text{“Perryridge”}))\}$$

- Find the names of all customers who have a loan, an account, or both at the Perryridge branch:

$$\{< c > \mid \exists\, l\, (< c, l > \in\ borrower \wedge \exists\, b, a\, (< l, b, a > \in\ loan\ \wedge\ b\ =\ \text{“Perryridge”})) \vee \exists\, a\, (< c, a > \in\ depositor \wedge \exists\, b, n\, (< a, b, n > \in\ account\ \wedge\ b\ =\ \text{“Perryridge”}))\}$$

- Find the names of all customers who have an account at all the branches located in Brooklyn:

$$\{< c > \mid \exists\, n\, (< c, n > \in\ customer) \wedge \forall\, x, y, z\, (< x, y, z > \in\ branch\ \wedge\ y\ =\ \text{“Brooklyn”}\ \Rightarrow \exists\, a, b\, (< a, x, b > \in\ account \wedge\ < c, a > \in\ depositor))\}$$

  In English, we interpret this expression as “The set of all (*customer-name*) tuples c such that, for all (*branch-name*, *branch-city*, *assets*) tuples, x, y, z, if the branch city is Brooklyn, then the following is true”:
  - ☐ There exists a tuple in the relation *account* with account number *a* and branch name x.
  - ☐ There exists a tuple in the relation *depositor* with customer *c* and account number *a*.”

3.7.3 Safety of Expressions

We noted that, in the tuple relational calculus (Section 3.6), it is possible to write expressions that may generate an infinite relation. That led us to define *safety* for tuple-relational-calculus expressions. A similar situation arises for the domain relational calculus. An expression such as

$$\{< l, b, a > \mid \neg(< l, b, a > \in\ loan)\}$$

is unsafe, because it allows values in the result that are not in the domain of the expression.

For the domain relational calculus, we must be concerned also about the form of formulae within “there exists” and “for all” clauses. Consider the expression

$$\{< x > \mid \exists\, y\, (< x, y > \in\ r)\ \wedge\ \exists\, z\, (\neg(< x, z > \in\ r)\ \wedge\ P(x, z))\}$$

where P is some formula involving x and z. We can test the first part of the formula, $\exists\, y\, (< x, y > \in\ r)$, by considering only the values in *r*. However, to test the second part of the formula, $\exists\, z\, (\neg\, (< x, z > \in\ r)\ \wedge\ P(x, z))$, we must consider values for z that do not appear in *r*. Since all relations are finite, an infinite number of values do not appear in *r*. Thus, it is not possible, in general, to test the second part of the

formula, without considering an infinite number of potential values for z. Instead, we add restrictions to prohibit expressions such as the preceding one.

In the tuple relational calculus, we restricted any existentially quantified variable to range over a specific relation. Since we did not do so in the domain calculus, we add rules to the definition of safety to deal with cases like our example. We say that an expression

$$\{< x_1,\ x_2, \ldots, x_n > \mid P\,(x_1,\ x_2, \ldots, x_n)\}$$

is safe if all of the following hold:

1. All values that appear in tuples of the expression are values from *dom*(*P*).
2. For every "there exists" subformula of the form $\exists\, x\ (P_1(x))$, the subformula is true if and only if there is a value *x* in $dom(P_1)$ such that $P_1(x)$ is true.
3. For every "for all" subformula of the form $\forall x\ (P_1(x))$, the subformula is true if and only if $P_1(x)$ is true for all values *x* from $dom(P_1)$.

The purpose of the additional rules is to ensure that we can test "for all" and "there exists" subformulae without having to test infinitely many possibilities. Consider the second rule in the definition of safety. For $\exists\, x\ (P_1(x))$ to be true, we need to find only one *x* for which $P_1(x)$ is true. In general, there would be infinitely many values to test. However, if the expression is safe, we know that we can restrict our attention to values from $dom(P_1)$. This restriction reduces to a finite number the tuples we must consider.

The situation for subformulae of the form $\forall x\ (P_1(x))$ is similar. To assert that $\forall x\ (P_1(x))$ is true, we must, in general, test all possible values, so we must examine infinitely many values. As before, if we know that the expression is safe, it is sufficient for us to test $P_1(x)$ for those values taken from $dom(P_1)$.

All the domain-relational-calculus expressions that we have written in the example queries of this section are safe.

3.7.4 Expressive Power of Languages

When the domain relational calculus is restricted to safe expressions, it is equivalent in expressive power to the tuple relational calculus restricted to safe expressions. Since we noted earlier that the restricted tuple relational calculus is equivalent to the relational algebra, all three of the following are equivalent:

- The basic relational algebra (without the extended relational algebra operations)
- The tuple relational calculus restricted to safe expressions
- The domain relational calculus restricted to safe expressions